

Rapport parallélisme

Nom: Bastien NGUYEN

Numéro étudiant: 22300553

Groupe: B11

Exercice 5



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Université
de Toulouse

Introduction

Pour rappel j'ai 6 coeur sur ma machine. Tout les résultats que je montre seront fait à partir de ma machine.

Voici la liste de tout les éléments dans mon archive:

- les 3 images de tests: image0.ppm, image1.ppm et image2.ppm.
- les dossiers de resultats fournies avec un ajout pour m'adapter à windows.
- un dossier obtained qui me sert à stocker les résultats obtenues via les différents programmes.
- un fichier ex5-seq qui est la version séquentielle demandée.
- un fichier ex5-par qui est la version parallèle demandée.
- un fichier ex5-moy qui me permet de comparer plusieurs itération de séquentielle et parallèle de façon à obtenir des résultats plus correctes.

Pour ce tp, j'ai eu un problème avec les fichiers de résultats fournies. En effet en passant des ordinateurs de la fac sous linux à ma machine sous windows il y avait une différence de format de fichier et donc je me suis créer un dossier avec les résultats adapté grâce au commandes dos2unix et unix2dos.

Pour mieux comprendre ex5-moy il prend 3 arguments l'image a traite, le nombre de thread max sur lequel le programme va tester la version parallèle et le nombre d'itération utilisé pour faire la moyenne. Plus tard pour mon fichier ex5-moy je me suis rajouté une option permettant de choisir avec quel dossier on vérifie que les calculs se soient bien effectué pour pouvoir me passer de la commandes diff.

Séquentielle

Pour faire la version séquentielle j'ai décidé de faire l'entièreté des calculs dans une seule et même fonction `greyToContrast`. Cela me permet de tout paralléliser dans une seule fonction. Pour les calculs du tableau `C`, j'ai remarqué que on pouvait faire: $C[i] = C[i-1] + H[i]$ afin d'accélérer le temps de calcul mais cela empêchera de paralléliser cette partie si on ne la modifie pas.

Pour les mesures de temps comme l'énoncé nous demande de paralléliser la fonction `colorToGrey` je vais donc mesurer le temps d'exécution de cette fonction en y ajoutant celui de ma propre fonction.

Voici les différentes mesures que j'ai obtenu pour cette version:

Image 0	image 1	image 2
0,00041s	0,0033s	0,057s

On voit bien que la différences des tailles des dimages influent bcp sur le temps de calcul. On fait environ un fois 100 entre l'image 0 et 2.

Parallèle

Pour réaliser la version parallèle, j'ai d'abord réfléchi à ce qui peut être paralléliser ou non.

Pour colorToGrey les deux for sont parallélisable car il n'y a aucune dépendance entre les itérations de la boucle, je vais donc paralléliser le premier.

Pour greyToContrast, le calcul de H est parallélisable cependant comme chaque thread peut incrémenter un même index de H il faut protéger H ce pourquoi je calcul d'abord quel index de H je vais incrémenter puis je fais une réduction sur le tableau de taille 256. Ensuite le calcul de C n'est pas parallélisable comme il apparaît en séquentiel cependant si je modifie la formule de façon à ne plus utiliser $C[i-1]$ alors il l'est. J'ai utilisé un schedule car on observe que les dernières itérations coûteront plus que les premières. Pour le coup, cela demande plus de calcul que la version séquentielle mais ces calculs sont accélérés donc on revient à ne pas accélérer.

Dans un temps normal, la décision aurait été de ne pas paralléliser ce calcul pour ne pas gaspiller de la puissance de calcul mais je l'ai laissé pour montrer que je l'avais vu.

Enfin le calcul de la nouvelle image est aussi parallélisable j'ai donc utilisé un for classique.

Grace aux mesures temporelles (pages suivantes), on remarque bien que l'accélération s'améliore jusqu'à mon nombre de coeur ou elle commence à stagner.

Parallèle

Nombre de threads	image 0	image 1	image 2
1	0,00059s	0,0061s	0,062s
2	0,000396s	0,0022s	0,035s
4	0,000256s	0,0012s	0,019s
6	0,000223s	0,001s	0,015s
8	0,000228s	0,0015s	0,013s
10	0,00039s	0,0013s	0,0127s
12	0,000547s	0,0016s	0,014s

Analyse

Grace aux mesures des deux versions on peut calculer l'accélération obtenu pour chaque image.

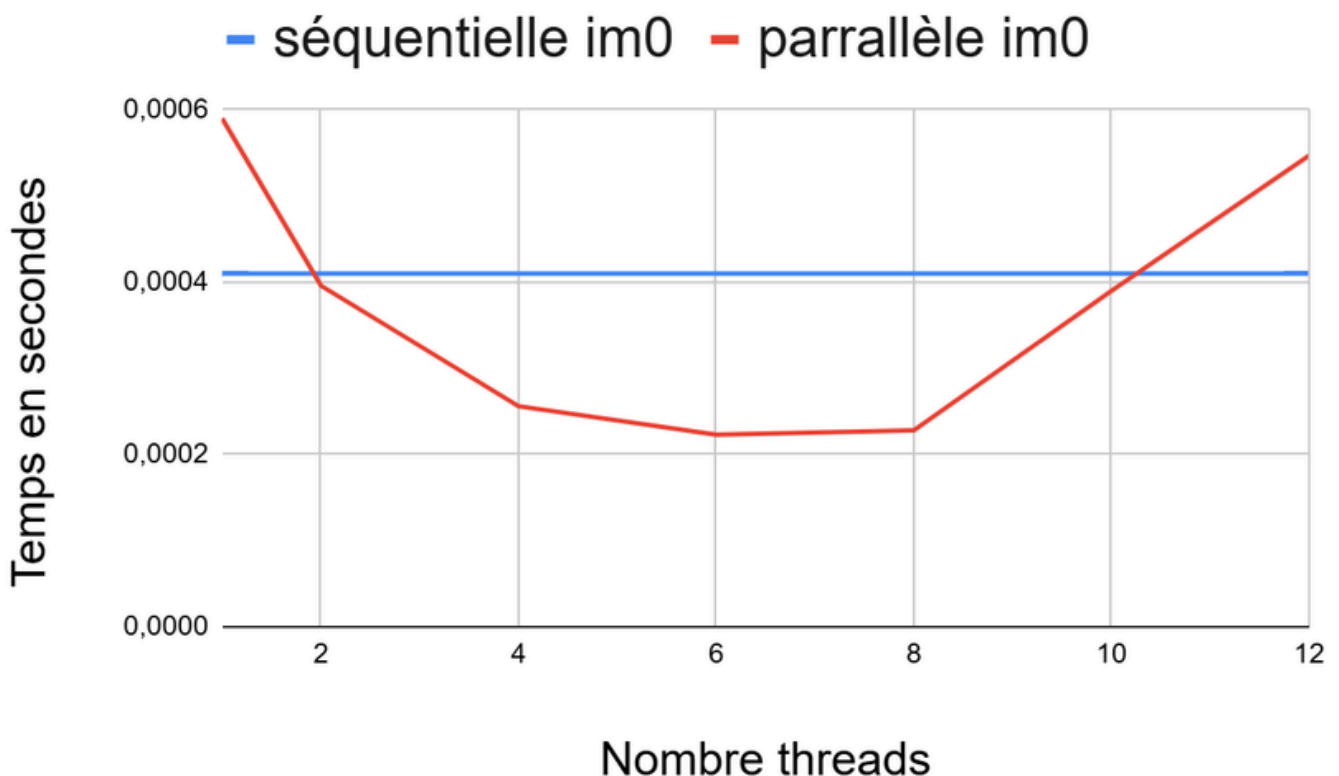
Voici un tableau des différentes accélération obtenue en fonction du nombre de threads.

Nombre de threads	image 0	image 1	image 2
1	0,69	0,54	0,92
2	1,04	1,50	1,63
4	1,60	2,75	3,00
6	1,84	3,30	3,80
8	1,05	2,20	4,38
10	1,05	2,54	4,49
12	0,75	2,06	4,07

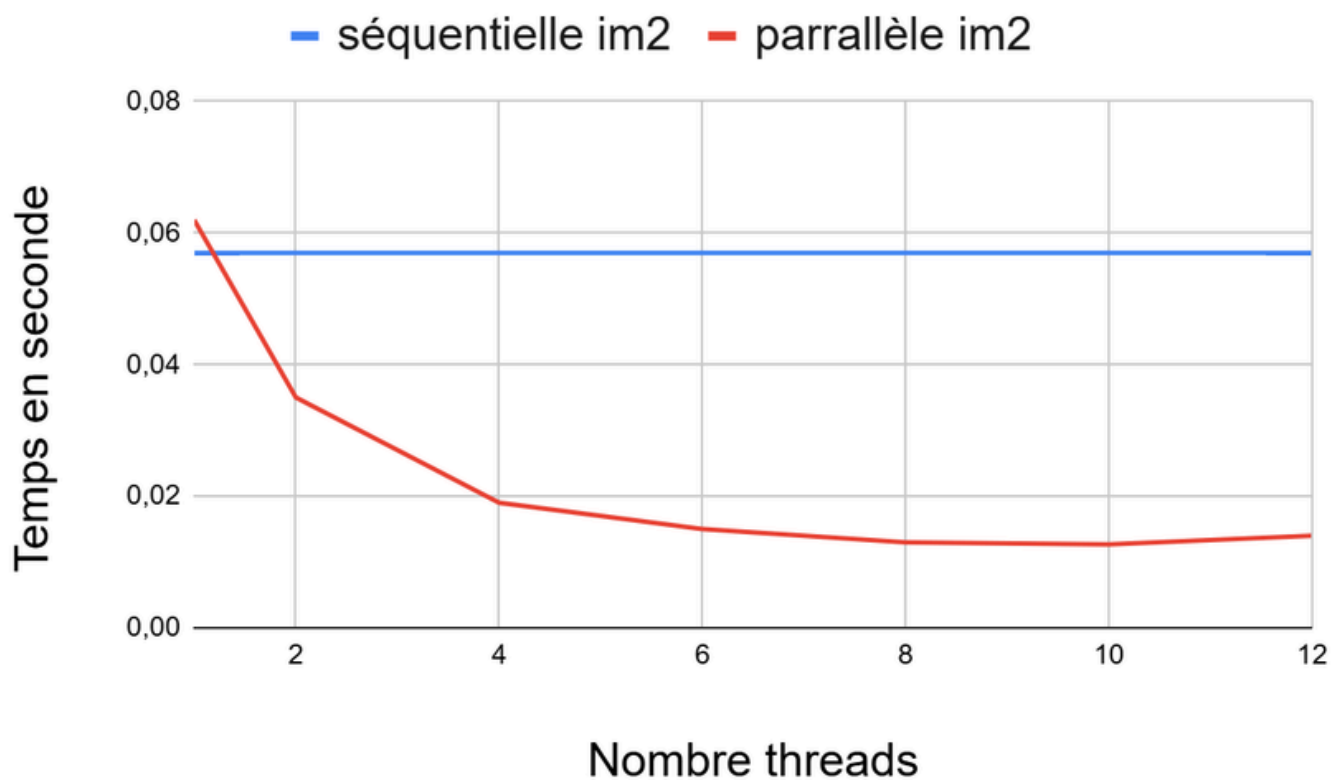
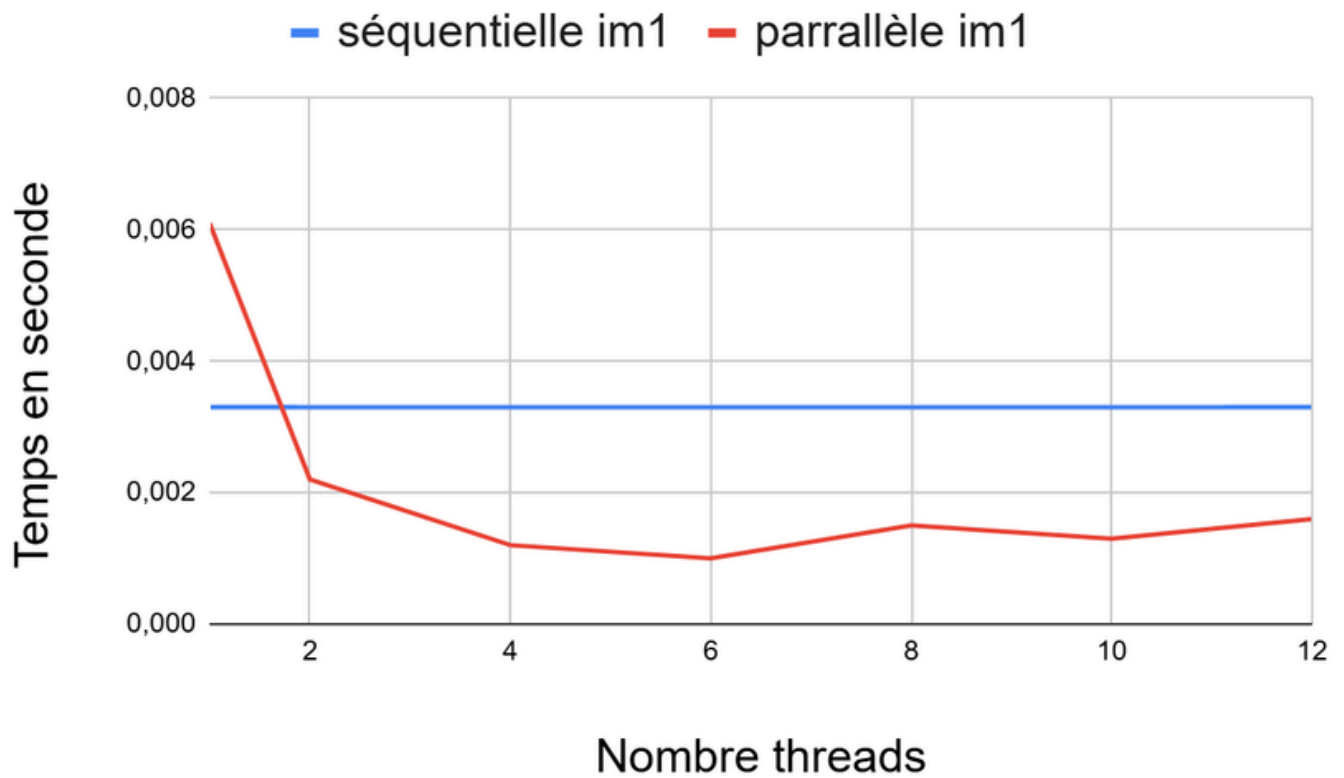
Analyse

En regardant les mesures d'accélération on peut conclure que plus il y a de calcul à faire plus la version parallèle est rentable mais qu'en dépassant mon nombre de coeur l'accélération ralentit puis même baisse avec trop de threads. Cela prouve donc bien que le coût des primitives d'OpenMp est rattrapé par le nombre de calcul mais que dans les cas où il y a peu de calcul comme l'image 0 alors on a une plus petite accélération. On peut ajouter que dans mes mesures temporelles il y a même un moment où le temps parallèle redevient plus grand que séquentielle avec 12 threads et l'image 0.

Pour mieux ce représenter cela voici des graphiques permettant de suivre l'évolution du temps d'exécution en fonction de l'image et du nombre de threads.



Analyse



Conclusion

Ce tp permet de bien comprendre les limites de la parallélisation. On observe bien que dépasser le nombre de coeur ralentit l'accélération voir l'inverse. On peut aussi remarquer que moins un programme feras de calcul moins le paralléliser seras rentable à cause du temps d'exécution des primitives OpenMp.

Malgré ces limites on comprend bien qu'une bonne utilisation du parallélisme permet de grandement accélérer des programmes fesant beaucoup de calcul (image 2).